

OpenMP

David Darras

20/05/2024

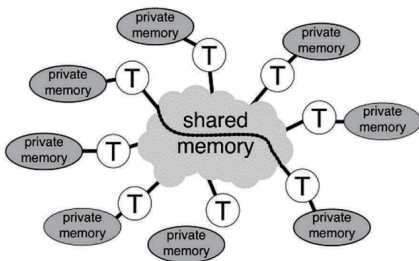
1 Récapitulatif

- **OpenMP** (= *Open Multi-Processing*) est une API en C conçue pour la parallélisation en mémoire partagée. Elle fournit des directives de compilation, des fonctions d'exécution et des variables d'environnement pour gérer ces opérations.

- Un **thread** est une unité d'exécution qui possède son propre contexte (c'est-à-dire une pile, un identifiant, etc.). Chaque thread a une mémoire privée à laquelle les autres threads n'ont pas accès, mais ils peuvent tous lire/écrire dans une mémoire partagée. (Et il est donc nécessaire de définir la portée des variables avec les directives OpenMP!)

Remarque : Les variables définies dans un bloc sont automatiquement privées. Les variables globales ou statiques sont par défaut partagées.

Attention : Deux threads ne peuvent pas écrire/lire sur le même emplacement mémoire simultanément ! Ils peuvent cependant créer une copie privée d'une variable partagée et la mettre à jour quand la ressource partagée sera disponible, par exemple.



- Un **accès concurrent** (ou *race condition*) se produit lorsqu'au moins deux threads tentent d'accéder et de modifier une ressource partagée simultanément. Cela entraîne des résultats imprévus car les opérations des threads peuvent interférer les unes avec les autres. Pour éviter cela, on utilise des opérations atomiques ou une exclusion mutuelle (mutex) sur une section critique.
- Une **section critique** désigne une partie du code où un thread peut accéder à une ressource partagée de manière exclusive, c'est-à-dire sans qu'aucun autre thread n'y accède simultanément.
- Une **opération atomique** ne peut pas être subdivisée en sous-opérations et est non interruptible. Elles sont

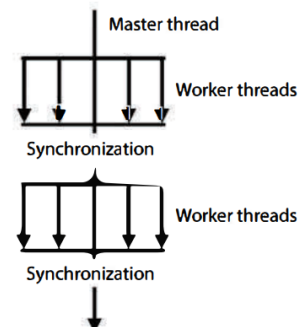
parfois préférées par le compilateur à l'égard d'une exclusion mutuelle pour des raisons de performances.

En ARM, on retrouve les instructions atomiques : LDREX, STREX, SWP et SWPB. Elles sont conçues pour fournir le comportement atomique requis pour les sémaphores sans verrouiller toutes les ressources du système entre les phases de lecture et d'écriture.

- L'**exclusion mutuelle** est le principe selon lequel une ressource partagée ne peut être utilisée que par un thread à la fois, garantissant ainsi l'intégrité des données et évitant les accès concurrents.
- Un autre défi important est celui de la **cohérence des données**. Lorsqu'un thread modifie une ressource partagée, les autres threads peuvent encore avoir accès à une version antérieure de cette ressource. Ainsi, il est nécessaire de mettre en place des synchronisations (avec un **flush**) pour garantir que tous les threads disposent d'une vue à jour de la mémoire partagée.
- **Structure** : L'utilisateur doit sélectionner les blocs où il souhaite appliquer une exécution en parallèle des instructions.

```
#pragma omp parallel [clause[[,]clause]...]
{
  ///< bloc
}
```

- **Principe** : Les instructions dans une région active sont exécutées par tous les threads, tandis que celles dans une région inactive ne seront exécutées que par le thread maître. Une barrière est placée à la fin de chaque région parallèle afin de synchroniser tous les threads et de redonner la main au thread maître lorsque tous les threads auront atteint la barrière.



2 Constructions et clauses

- La clause `num_threads(<n>)` permet de spécifier combien de threads doivent travailler sur le bloc.

Remarque : On peut aussi modifier la variable d'environnement `OMP_NUM_THREADS` ou utiliser la fonction `omp_set_num_threads` dans une région inactive!

- Les clauses `private(var1, var2, etc.)`, `firstprivate(...)` et `shared(...)` permettent de définir la portée de chaque variable. De plus, la clause `default(none)` force l'utilisateur à spécifier ces portées pour chaque variable utilisée dans le bloc.

```
/// Exemple n°1
int w = 1, x = 5, y = 20;
#pragma omp parallel private(x) firstprivate(y)
{
    /// x n'est pas initialisée à 5 ici
    /// // mais y a été pré-initialisée à 20
    x = 10;
    int z = x + y; /// z est aussi privée
    y = 30;
    w = 2; /// w est partagée
}
// Fin de la région parallèle
// w = 2, x = 5, y = 20
```

```
/// Exemple n°2
int sum = 0;
#pragma omp parallel shared(sum)
{
    int s; /// somme partielle privée du thread
    /// // Le thread calcule une valeur pour s

    /// // Sans la région critique,
    /// // ce code présente un accès concurrent
    #pragma omp critical
    {
        sum += s;
    }
}
```

Remarques: La section critique permet d'empêcher un autre thread de lire/écrire la valeur de `sum` avant que les opérations dans la section critique ne soient terminées. Or, après avoir mis à jour `sum`, rien ne garantit que les autres threads ont la nouvelle valeur pour `sum`. Pour s'en assurer, il faut utiliser la clause `flush` afin de garantir que tous les threads ont la même vue cohérente des données partagées. Or, celui-ci est implicite à l'entrée/sortie d'une section critique et d'une région parallèle (ou avec les fonctions `omp_set_lock` et `omp_unset_lock`).

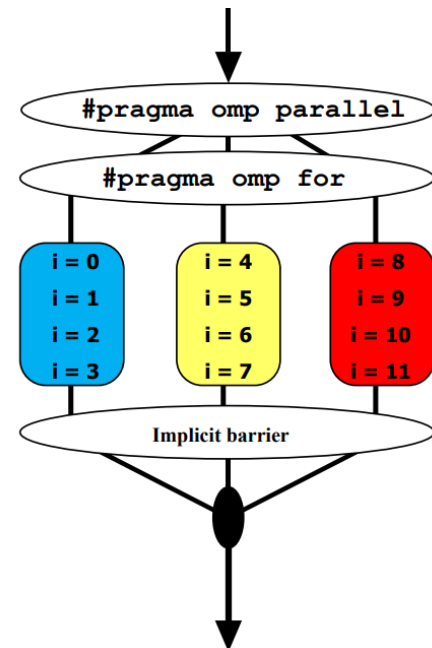
NB : `flush` ne met à jour que le thread courant.

- La construction de boucle `#pragma omp for` permet de découper une boucle itérative selon une planification donnée en plusieurs morceaux et de les partager

entre les différents threads.

Remarque : `#pragma omp ordered` peut être utilisé pour maintenir l'ordre séquentiel de l'exécution. Et, la clause `collapse(<n>)` permet de fusionner `n` boucles imbriquées en une seule boucle, ce qui améliore les performances.

- La clause `schedule` affecte la manière dont les itérations de la boucle sont réparties entre les threads. `static(<n>)` répartit de manière équitable les blocs d'itérations de taille `n` à chaque thread tandis que `dynamic(<n>)` donne un bloc d'itération de taille `n` au thread qui a terminé sa tâche.



- La construction `section` permet d'exécuter des blocs en parallèle.

```
#pragma omp parallel sections
{
    #pragma omp section
    {
    }
    #pragma omp section
    {
    }
    #pragma omp section
    {
    }
}
```

- Les constructions pour exécuter une portion de code par un seul thread sont :

```
#pragma omp single
/// Seul un thread exécute le bloc
/// // + barrière implicite

#pragma omp master
/// Garantit que le thread maître exécute le bloc
/// // Mais attention aux courses de données!
```

- Imbrication de parallélisme

```
#pragma omp parallel num_threads(3)
{
    // Code exécuté par 3 threads
    #pragma omp parallel num_threads(2)
    {
        // Code exécuté par 6 threads
    }
}
```

Pensez à autoriser l'imbrication en modifiant la variable d'environnement : OMP_NESTED ou en utilisant la fonction `omp_set_nested` !

- Les constructions de synchronisation sont :

- `#pragma omp barrier` : Pose une barrière pour éviter qu'un thread n'accède prématurément à des données initialisées par un autre thread, par exemple. Il faut attendre que tous les threads aient atteint la barrière avant de continuer.
- `#pragma omp critical` : Définit une section critique qui est un bloc exécuté par tous les threads, mais il est garanti qu'un seul thread à la fois peut être actif dans la région. Il n'y a pas de barrière implicite mais un flush implicite. On l'utilise pour mettre à jour des variables partagées et éviter la concurrence.
- `#pragma omp atomic` : Ne peut être utilisé que pour des opérations atomiques - donc pour mettre à jour une variable avec les opérations : `+`, `-`, `*`, `/`, `&`, `,`, `<<`, `>>`.

Clause	Expr. STMT (C/C++)
read	<code>v = x;</code>
write	<code>x = expr;</code>
update	<code>x++;</code> <code>++x;</code> <code>x--;</code> <code>--x;</code> <code>x binop = expr;</code> <code>x = x binop expr;</code> <code>x = expr binop x;</code>

Clause	Expr. STMT (C/C++)
capture	<code>v = x++;</code> <code>v = ++x;</code> <code>v = x--;</code> <code>v = --x;</code> <code>v = x binop = expr;</code> <code>v = x = x binop expr;</code> <code>v = x = expr binop x;</code>

- La construction d'annulation `#pragma omp cancel` permet d'arrêter tous les threads si le résultat d'une opération a été trouvé ou en cas d'erreur, un peu comme une exception en Java.

```
#pragma omp parallel
{
    char* buffer;
    buffer = (char*)malloc( 10 );
    if( buffer == NULL )
```

```
{
    #pragma omp cancel parallel
}
}
```

- La clause de réduction permet de spécifier des opérations de réduction sur des variables partagées. Les opérations de réduction sont l'addition, la multiplication, etc., qui sont effectuées de manière concurrente sur différentes parties des données et dont le résultat final est agrégé en une seule valeur.

Operator	Initializer	Combiner
<code>+</code>	<code>omp_priv = 0</code>	<code>omp.out += omp.in</code>
<code>*</code>	<code>omp_priv = 1</code>	<code>omp.out *= omp.in</code>
<code>-</code>	<code>omp_priv = 0</code>	<code>omp.out -= omp.in</code>
<code>&</code>	<code>omp_priv = ~0</code>	<code>omp.out &= omp.in</code>
<code> </code>	<code>omp_priv = 0</code>	<code>omp.out = omp.in</code>
<code>^</code>	<code>omp_priv = 0</code>	<code>omp.out ^= omp.in</code>
<code>&&</code>	<code>omp_priv = 1</code>	<code>omp.out = omp.in && omp.out</code>
<code> </code>	<code>omp_priv = 0</code>	<code>omp.out = omp.in omp.out</code>
<code>max</code>	<code>omp_priv = Least(T)</code>	<code>omp.out = omp.in > omp.out ? omp.in : omp.out</code>
<code>min</code>	<code>omp_priv = Largest(T)</code>	<code>omp.out = omp.in < omp.out ? omp.in : omp.out</code>
<code>+</code>	<code>omp_priv = 0</code>	<code>omp.out = omp.in + omp.out</code>
<code>*</code>	<code>omp_priv = 1</code>	<code>omp.out = omp.in * omp.out</code>
<code>-</code>	<code>omp_priv = 0</code>	<code>omp.out = omp.in + omp.out</code>
<code>.and.</code>	<code>omp_priv = .true.</code>	<code>omp.out = omp.in .and. omp.out</code>
<code>.or.</code>	<code>omp_priv = .false.</code>	<code>omp.out = omp.in .or. omp.out</code>
<code>.eqv.</code>	<code>omp_priv = .true.</code>	<code>omp.out = omp.in .eqv. omp.out</code>
<code>.neqv.</code>	<code>omp_priv = .false.</code>	<code>omp.out = omp.in .neqv. omp.out</code>
<code>max</code>	<code>omp_priv = Least(T)</code>	<code>omp.out = max(omp.in, omp.out)</code>
<code>min</code>	<code>omp_priv = Largest(T)</code>	<code>omp.out = min(omp.in, omp.out)</code>
<code>iand</code>	<code>omp_priv = not(0)</code>	<code>omp.out = iand(omp.in, omp.out)</code>
<code>ior</code>	<code>omp_priv = 0</code>	<code>omp.out = ior(omp.in, omp.out)</code>
<code>ieor</code>	<code>omp_priv = 0</code>	<code>omp.out = ieor(omp.in, omp.out)</code>

```
///< Exempl n°1
#pragma omp parallel for default(none) \
firstprivate(n) shared(a,b) reduction(+:sum)
for ( int i = 0; i < n; i++ )
    sum += a[i] * b[i];
```

```
///< Exempl n°2
///< Pour créer sa propre fonction
#pragma omp declare reduction(myAbsMin : int : \
omp_out = abs(omp_in) < omp_out ? abs(omp_in) : omp_out)
initializer(omp_priv = INT_MAX)
int result = INT_MAX;
#pragma omp parallel for default(none) \
firstprivate(n) shared(a) reduction(myAbsMin:res)
for (int i = 0; i < n; i++) {
    if (abs(a[i]) < res) {
        res = abs(a[i]);
    }
}
```

- La clause `nowait` permet de supprimer la barrière de synchronisation implicite, ce qui améliore les performances car les threads ne doivent pas attendre les uns les autres. Cependant, cela peut entraîner des problèmes de cohérence.

3 Référence

- Beaucoup de choses ont été omises dans cette fiche, telles que les tâches, SIMD, la cohérence mémoire

relâchée, etc. Je vous invite donc à lire *Using OpenMP – The Next Step* si vous souhaitez en apprendre davantage.

